**UNIVERSIDAD TECNOLÓGICA NACIONAL**
Tecnicatura Universitaria en Programación a Distancia

Programación 1

# Trabajo Práctico Integrador

## Sistema de Control de Inventario

*Informe de diseño y fundamentación*

**Alumna:** Hidalgo Abril Antonella
**Fecha:** noviembre de 2026

# 1. Introducción

En este informe explicamos cómo diseñamos el sistema de control de inventario para una ferretería. El programa permite cargar herramientas con su stock, consultarlas, ver cuáles están agotadas, dar de alta productos nuevos y registrar ventas e ingresos de mercadería. Todo se maneja desde un menú que se repite hasta que el usuario elige salir, y cada operación actualiza el stock en el momento.

Además de que el programa funcione, buscamos que resista los errores comunes del usuario: textos donde se espera un número, nombres vacíos o repetidos, cantidades negativas o ventas mayores al stock disponible. En las secciones siguientes vinculamos cada concepto de la materia con la parte del código donde lo aplicamos, presentamos los diagramas de flujo y describimos el entorno de desarrollo.

# 2. Conceptos aplicados

## 2.1 Listas

Una lista es una colección ordenada de elementos que puede crecer mientras el programa se ejecuta. En nuestro programa toda la información vive en una única lista llamada `inventario`, que se crea vacía al comienzo de `main()`. Cada vez que se carga una herramienta (opción 1) o se da de alta una nueva (opción 5), la agregamos al final con `append()`. Para saber si todavía no hay datos usamos `len(inventario) == 0`, y así las opciones 2, 3, 4 y 6 avisan que no hay herramientas cargadas sin pedir ningún dato.

La lista no se declara como variable global: la creamos dentro de `main()` y se la pasamos por parámetro a cada función del menú. Como las listas son mutables, la función recibe una referencia a la misma lista y no una copia, por eso lo que agrega `cargar_herramientas()` se ve después en todas las demás opciones.

```
def main():
    inventario = []
    ...
    cargar_herramientas(inventario)
```

## 2.2 Diccionarios

Un diccionario guarda datos en pares clave–valor, lo que permite acceder a cada dato por su nombre. Cada herramienta del inventario es un diccionario con exactamente dos claves: `'herramienta'`, que guarda el nombre, y `'cantidad'`, que guarda el stock. El diccionario lo armamos directamente dentro del `append()`, sin crear un diccionario suelto antes:

```
inventario.append({'herramienta': nombre, 'cantidad': stock})
```

Para leer o modificar el stock accedemos por clave. Por ejemplo, en la opción 6 una venta hace `producto['cantidad'] = producto['cantidad'] - unidades`. Como `producto` apunta a un diccionario que ya está dentro de la lista, el cambio queda guardado en el inventario.

## 2.3 Funciones

Una función agrupa un bloque de código con una tarea concreta, se define con `def` y puede recibir parámetros y devolver un resultado con `return`. Dividimos el programa en funciones para que cada una haga una sola cosa y sea fácil de leer y de corregir. Cada opción del menú, de la 1 a la 6, tiene su propia función, y la opción 7 se resuelve en `main()` cuando termina el bucle del menú.

| Función | Qué hace |
|---|---|
| `convertir_entero(texto)` | Convierte un texto a entero; si no se puede, lanza un error con un mensaje claro. |
| `validar_nombre(nombre)` | Quita los espacios de los extremos y controla que el nombre no quede vacío. |
| `buscar_herramienta(inventario, nombre)` | Recorre el inventario y devuelve la herramienta encontrada, o None. |
| `validar_nombre_nuevo(inventario, nombre)` | Valida el nombre y controla que no esté repetido. |
| `validar_stock_inicial(texto)` | Convierte el stock y controla que no sea negativo (el 0 es válido). |
| `cargar_herramientas(inventario)` | Opción 1: carga múltiple de herramientas con su stock inicial. |
| `mostrar_inventario(inventario)` | Opción 2: lista todas las herramientas con su stock. |
| `consultar_stock(inventario)` | Opción 3: busca una herramienta y muestra su stock. |
| `reporte_agotados(inventario)` | Opción 4: muestra solo las herramientas con stock cero. |
| `alta_producto(inventario)` | Opción 5: agrega una herramienta nueva al final del inventario. |
| `actualizar_stock(inventario)` | Opción 6: registra una venta o un ingreso de unidades. |
| `mostrar_menu()` | Imprime las siete opciones del menú. |
| `main()` | Crea el inventario y repite el menú hasta que se elige la opción 7. |

*Tabla 1. Funciones del programa y su responsabilidad.*

Las funciones auxiliares las armamos para no repetir código: por ejemplo, la opción 1 y la opción 5 necesitan exactamente las mismas validaciones de nombre y de stock, así que las dos usan `validar_nombre_nuevo()` y `validar_stock_inicial()`. Todos los nombres usan snake_case.

## 2.4 Condicionales

Los condicionales (`if`, `elif`, `else`) permiten que el programa tome caminos distintos según una condición. Los usamos en tres lugares principales:

• **En el menú:** una cadena de `if/elif` en `main()` decide qué función ejecutar según la opción elegida. La última rama, `elif opcion != 7`, lanza un error si el número está fuera del rango de 1 a 7.

• **En las reglas de negocio:** antes de modificar datos comprobamos condiciones como `stock < 0`, `unidades <= 0` o `producto['cantidad'] - unidades < 0`. Si alguna se cumple, se lanza un error.

• **En los casos informativos:** el inventario vacío, una herramienta que no existe (`producto is None`) o un tipo de movimiento que no es `'1'` ni `'2'` se resuelven con un `if` y un mensaje, sin modificar nada.

## 2.5 Bucles

Los bucles repiten un bloque de código. Usamos `while` cuando no sabemos cuántas veces se va a repetir y `for` cuando recorremos algo o repetimos una cantidad conocida de veces:

• **Menú persistente:** un `while opcion != 7` en `main()` vuelve a mostrar el menú después de cada operación.

• **Reintentos de la opción 1:** tres bucles `while True` vuelven a pedir la cantidad de herramientas, el nombre o el stock hasta que el dato sea válido; cuando lo es, `break` sale del bucle.

• **Carga múltiple:** `for i in range(cantidad_a_cargar)` repite la carga tantas veces como indicó el usuario.

• **Recorridos del inventario:** `for producto in inventario` aparece en el listado, en la búsqueda y en el reporte de agotados. En el reporte usamos una variable `hay_agotados` para saber si encontramos alguno y mostrar un mensaje distinto cuando ninguna herramienta tiene stock cero, sin crear otra lista.

## 2.6 Manejo de errores con try/except y raise

Distinguimos dos situaciones distintas. La primera es una **entrada inválida**: el usuario escribe algo que no se puede convertir a entero, como "abc" o "3.5". En ese caso `int()` ya produce un `ValueError`. Lo capturamos con `try/except` dentro de `convertir_entero()` y lo volvemos a lanzar con un mensaje en castellano:

```
def convertir_entero(texto):
    try:
        return int(texto)
    except ValueError:
        raise ValueError(f"'{texto}' no es un número entero.")
```

La segunda es una **regla de negocio**: el dato es válido como texto o número, pero no respeta lo que pide el sistema (nombre vacío o duplicado, stock negativo, cantidades menores o iguales a 0, venta mayor al stock u opción fuera de rango). Ahí comprobamos la condición con un `if` y lanzamos el error con `raise ValueError` y un mensaje descriptivo.

Lo más importante del diseño es **dónde se captura cada error**:

• **Opción 1:** los errores se capturan dentro de `cargar_herramientas()`, porque hay que volver a pedir solo el dato que falló sin perder la carga. Si el stock es inválido, se pide de nuevo solo el stock y se conserva el nombre.

• **Opciones 3, 5 y 6:** la función no captura el error. El error sale de la función y llega al `except` de `main()`, que muestra el motivo con `print(f'Error: {e}')` y vuelve al menú.

• **Menú:** el mismo `except` de `main()` atiende la opción que no es un número y la que está fuera de rango.

Gracias a que cada validación ocurre antes de modificar los datos, ningún dato inválido deja el inventario a medias: una herramienta se agrega con `append()` recién cuando su nombre y su stock son válidos, y el stock solo cambia en la opción 6 después de pasar todas las comprobaciones.

## 3. Diagramas de flujo

Los siguientes diagramas muestran el flujo del menú principal y los recorridos de carga, validación de datos y actualización de stock, tal como están implementados en el código. Los rombos amarillos son decisiones, los recuadros rojos son errores y las flechas punteadas indican un error que sale de la función hacia main().

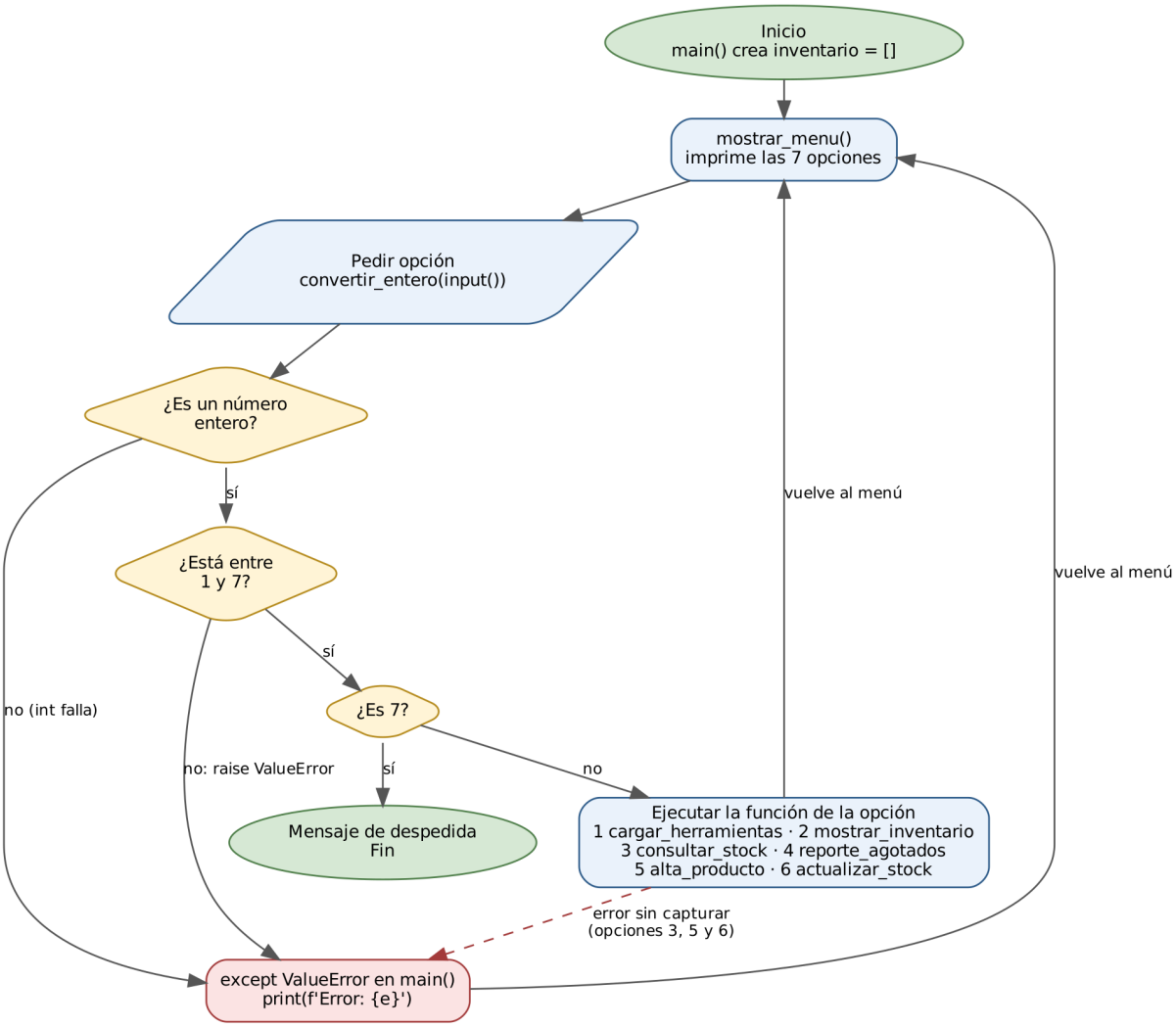



*Figura 1. Menú principal y manejo de errores en main().*

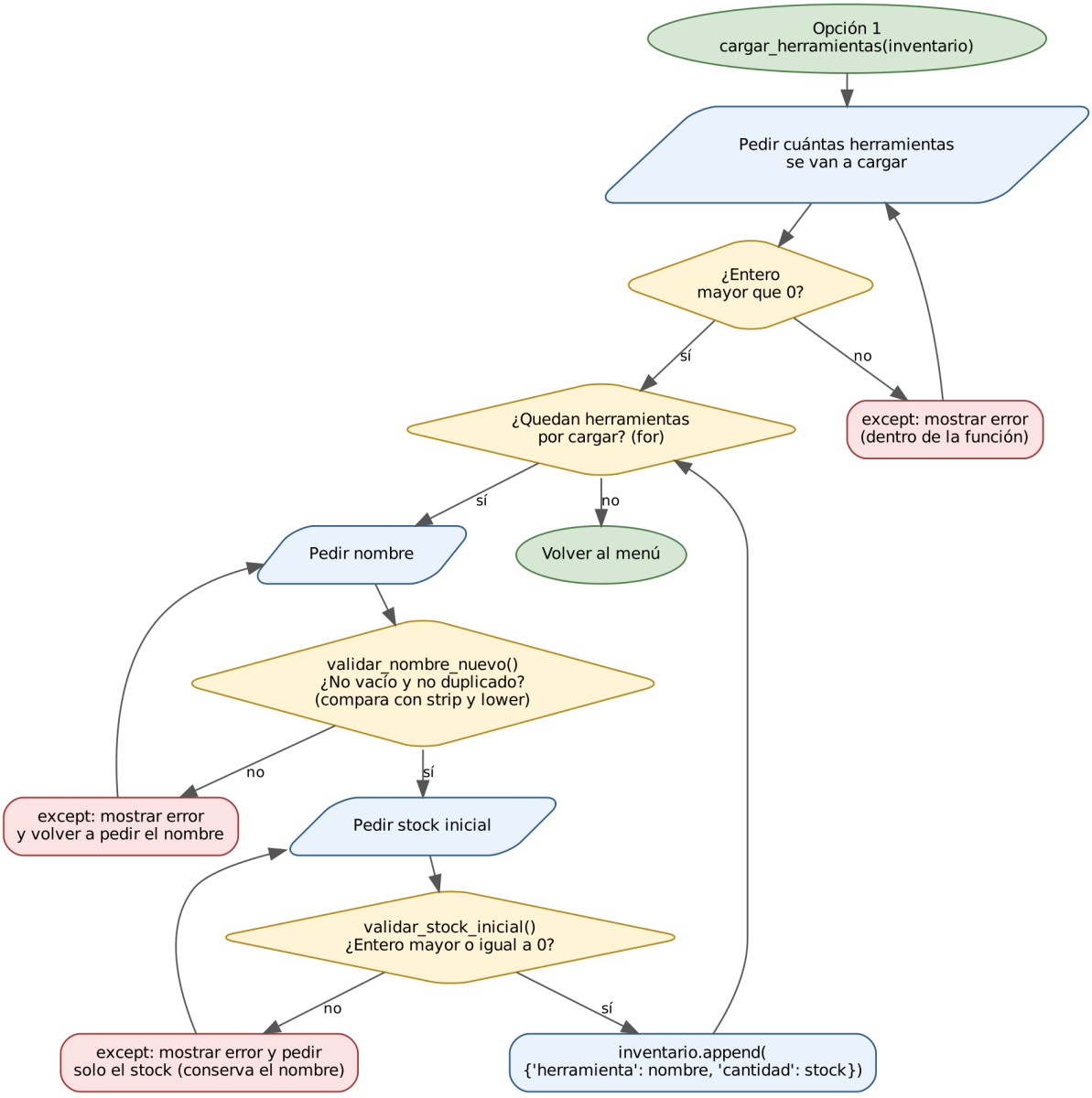



*Figura 2. Opción 1: carga de herramientas con validación y reintento dentro de la función.*

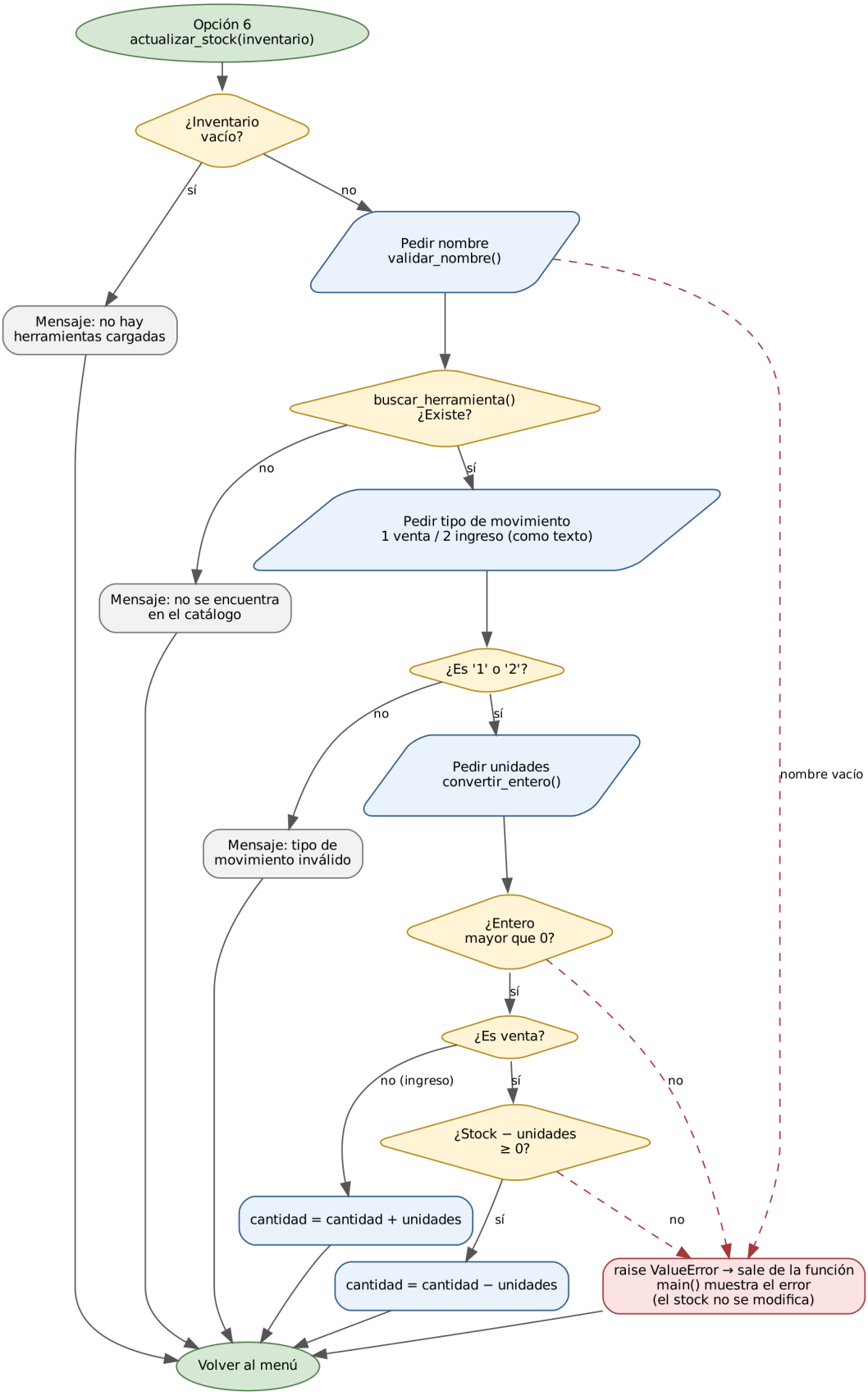



*Figura 3. Opción 6: actualización de stock (venta o ingreso).*

## 4. Lenguaje, entorno y recursos utilizados

**Python.** Es un lenguaje de programación interpretado, de propósito general y con una sintaxis clara, que se usa mucho para aprender a programar. Usamos la versión Python 3.14.3. Lo elegimos porque es el lenguaje de la materia y porque trae incorporado todo lo que necesitamos para este trabajo: listas, diccionarios, funciones y la excepción `ValueError`, sin instalar bibliotecas externas.

**Visual Studio Code.** Es un editor de código gratuito desarrollado por Microsoft. Usamos la versión 1.145.2. Lo elegimos porque resalta la sintaxis, marca errores mientras escribimos y tiene una terminal integrada, así que podemos escribir y probar el programa en el mismo lugar. Además es el editor que pide el enunciado para mostrar el código en el video.

**Extensión de Python para VS Code.** Es la extensión oficial de Microsoft que agrega a VS Code el soporte para Python: autocompletado, detección de errores y la posibilidad de ejecutar el archivo directamente. La elegimos porque facilita encontrar errores de escritura antes de ejecutar el programa.

**Terminal integrada.** Es la consola que trae VS Code. Desde ahí ejecutamos el programa y probamos todos los casos de la tabla de pruebas manuales del enunciado, tanto los de éxito como los de error.

**Cómo se ejecuta.** Todo el programa está en un único archivo, `inventario.py`. Para iniciarlo se abre una terminal en la carpeta del proyecto y se escribe `python inventario.py`.